RV College of Engineering®

(Autonomous Institution affiliated to VTU, Belagavi)

Department of Information Science and Engineering

Bengaluru - 560 059

Lab Manual for IV Semester B.E

INTERNET OF THINGS & APPLICATIONS

(IS244AI)

AY: 2025-26

Faculty In-charge

Dr. G.S. Mamatha
Professor and Head
Department of ISE

Prof. B K Srinivas
Assistant Professor
Department of ISE

RV College of Engineering®

(Autonomous Institution Affiliated to VTU, Belagavi)

Department of Information Science & Engineering

Bengaluru – 560059

LABORATORY CERTIFICATE

This is to certify that Mr./Ms. ________________________________

USN No: ________________________Semester: ____________________

has satisfactorily completed the course of experiments in Practical Internet of Things and Applications prescribed by the Department of Information Science and Engineering during the year 2025-26.

Marks	
Maximum	**Obtained**

Signature of the Faculty in-charge **Head of the Department**

RV College of Engineering®
Bengaluru - 560 059

Department of Information Science and Engineering

Vision

To be the hub for innovation in Information Science & Engineering through Teaching, Research, Development and Consultancy; thus, make the department a well-known resource centre in advanced, sustainable and inclusive technology.

Mission

ISE1: To enable students to become responsible professionals, strong in fundamentals of information science and engineering through experiential learning.
ISE2: To bring research and entrepreneurship into class rooms by continuous design of innovative solutions through research publications and dynamic development-oriented curriculum.
ISE3: To facilitate continuous interaction with the outside world through student internship, faculty consultancy, workshops, faculty development programmes, industry collaboration and association with the professional societies.
ISE4: To create a new generation of entrepreneurial problem solvers for a sustainable future through green technology with an emphasis on ethical practices, inclusive societal concerns and environment.
ISE5: To promote team work through inter-disciplinary projects, co-curricular and social activities.

Program Educational Objective (PEOs)

PEO1: To provide adaptive and agile skills in Information Science and Engineering needed for professional excellence / higher studies /Employment, in rapidly changing scenarios
PEO2: To provide students a strong foundation in basic sciences and its applications to technology
PEO3: To train students in core areas of Information science and Engineering, enabling them to analyse, design and create products and solutions for the real-world problems, in the context of changing technical, financial, managerial and legal issues.
PEO4: To inculcate leadership, professional ethics, effective communication, team spirit, multidisciplinary approach in students and an ability to relate Information Engineering issues to social and environmental context.
PEO5: To motivate students to develop passion for lifelong learning, innovation, career growth and professional achievement.

Program Outcomes:

Engineering Graduates will be able to:

PO1. **Engineering knowledge**: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.

PO2. **Problem analysis**: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.

PO3. **Design/development of solutions**: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.

PO4. **Conduct investigations of complex problems**: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

PO5. **Modern tool usage**: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modelling to complex engineering activities with an understanding of the limitations.

PO6. **The engineer and society**: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

PO7. **Environment and sustainability**: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.

PO8. **Ethics**: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

PO9. **Individual and team work**: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.

PO10. **Communication**: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

PO11. **Project management and finance**: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

PO12. Life-long learning: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

Laboratory Requirements

Prerequisites: Following open source software are compulsory requirements

1. Arduino IDE Latest version.
2. Putty software (for remote connectivity)
3. VNC Viewer and Server softwares (for remote connectivity)
4. User account on "ThingsSpeak" platform (Cloud platform for hosting IoT Applications)

Experiments List

Note: The following experiments can be conducted using C/Python language.

1. Set up a **timer** and delay for each of the three **LED lights** using Arduino
2. Interface **Pulse Sensor** with **Arduino** to Measure **Pulse Rate (BPM)** or **Heart Beat** value. Pulse rate will be displayed on 16×2 **LCD Display**
3. Interface **PC** with Arduino Uno using **Serial Communication Port** to capture different kinds of information on **serial monitor**, including system stats like network utilization, CPU load and disk space.
4. Interface **gas detector** with Arduino to switch on the **buzzer** on gas leakage.
5. Demonstrate the use of **accelerometer** by measuring the angle of tilt on **serial monitor** based on object motion with Arduino
6. Demonstrate object detection using **PIR sensor** and **LED interfacing** with Raspberry-Pi.
7. Demonstrate the measuring of Temperature & Humidity of weather using the **DHT11 Sensor** and display on 16 X 2 LCD Display interfacing with the Raspberry-Pi. Also share the data using webserver at the same time to display the weather information on the browser.
8. **Water level sensing** and sending water level information to **email** using **SMTP library** on Raspberry-Pi.
9. Develop a Smart lighting application using **LDR sensor** interfaced with Raspberry-Pi, **connected to cloud** for usage analysis.
10. Demonstrate the use of **soil moisture sensor** by alerting the user through a **mail/message** based on moisture level with Raspberry-Pi

Course Outcomes

After completing the course, the students will be able to

CO1: Understand the fundamentals of direct integration of the physical world with computer based systems.
CO2: Design & Implement solutions for Internet of Things with Raspberry Pi and Arduino through basic knowledge of programming and interfacing of input/output devices
CO3: Apply and analyse the analog & digital data with advanced interfacing techniques
CO4: Create visualizations for IoT data captured through real time systems to help decision making systems.

Rubrics for Evaluation of Experiments

Sl. No.	Criteria	Measuring Method	Excellent	Good	Poor
Lab Write-up and Execution rubrics (Max: 6 marks)					
a.	**Design and Implementation of IoT Solution (CO2)**	Design and Implementation skills	Student demonstrates a clear understanding of IoT requirements and successfully designs and implements a solution using appropriate IoT sensors and technologies (2 Marks)	Student exhibits an understanding of IoT requirements and implements a solution using suitable IoT protocols and technologies (1-1.5 Marks)	Student lacks understanding of IoT requirements and struggles to implement a solution using appropriate IoT protocols and technologies (0 Marks)
b.	**Integration and circuit rig up of sensors for IoT Solution (CO3)**	Integration skills	IoT solution integrates components and is tested under various conditions, yielding reliable results (2 Marks)	IoT solution integrates components and is tested under standard conditions, delivering acceptable results (1-1.5 Marks)	No integration and testing of the IoT solution (0 Marks)
c.	**Results and Documentation of IoT Solution (CO4)**	Presentation and Documentation skills	Comprehensive documentation showcases all aspects of the IoT solution, including changes made and results obtained, in detailed reports (2 Marks)	Documentation presents results adequately in reports (1-1.5 Marks)	Lack of proper result presentation and documentation (0 Marks)
Rubrics for viva voce (Max: 4 marks)					
a.	**Conceptual Understanding of IoT Technologies and Applications (CO1)**	Viva Voice	Provides a comprehensive explanation of IoT technologies and their applications in solving experimental problems (2 Marks)	Offers an explanation on IoT technologies and their applications in solving experimental problems (1-1.5 Marks)	Unable to explain concepts related to IoT technologies (0 Marks)
b.	**Practical Application of IoT Concepts in the Experiments (CO2)**	Viva Voice	Demonstrates the use of IoT concepts and strategies to address challenges in experimental setups (1 Mark)	Shows an application of IoT concepts to address challenges in experimental setups (0.5 Mark)	Fails to apply IoT concepts effectively in experimental setups (0 Marks)
c.	**Communication and Clarity of Presentation (CO1)**	Viva Voice	Communicates all ideas and demonstrations clearly, facilitating understanding among the audience (1 Mark)	Communicates ideas and demonstrations moderately well (0.5 Mark)	Unable to communicate ideas and demonstrations clearly (0 Marks)

Rubrics for Project Evaluation					
Rubrics for Phase-1 evaluation					
a	**Project Concept & Innovation**	Clear, unique, and highly innovative idea. (5 M)	Clear idea with some innovative aspects. Problem addressed is relevant. (4 M)	Idea lacks clarity or innovation. Problem is somewhat relevant but not fully addressed. (3 M)	Project idea is vague, uninspiring, or not relevant to current issues.(1/2 M)
b	**Technical Design & Architecture**	Highly functional, scalable, and robust system design. (10 M)	Functional design with good integration of components. Some minor scalability concerns. (8/9 M)	Adequate design, but with significant issues in integration, scalability, or robustness. (5-7 M)	Poorly designed system with major issues in integration or scalability. (1-4 M)
c	**Sensor/Device Selection & Integration**	Excellent choice and integration of sensors/devices, with full consideration for performance and cost. (10 M)	Good choice and integration, with minor issues in performance or cost-efficiency. (7-9 M)	Fair choice of sensors/devices; lacks optimization in performance or cost. (4-6 M)	Poor selection and integration of devices, with significant performance issues.(1-3 M)
Rubrics for Phase-2 evaluation					
a	**User Interface (UI) & Experience (UX)**	Intuitive, visually appealing, and easy-to-navigate interface (5 M)	Good interface with minor usability or aesthetic issues. (4 M)	Interface is functional, but not user-friendly or aesthetically pleasing. (2-3 M)	Difficult or confusing interface that hinders user experience. (1 M)
b	**Testing & Validation**	Extensive testing in diverse conditions (10 M)	Sufficient testing with reasonable results; some areas under-tested. (7-9 M)	Basic testing performed; some significant areas left untested. (4-6 M)	Very little or no testing conducted. (1-3 M)
c	**Documentation & Reporting**	Complete and professional documentation, including code, system design, and test results. (10 M)	Good documentation with most necessary information but lacks detail in some areas. (8-9 M)	Adequate documentation but missing key details, such as code or design files. (4-6 M)	Incomplete or unclear documentation, hard to follow or understand. (1-3 M)

Deliverables for Phase – 1 Evaluation: Project abstract, Existing system survey, System-Design/Architecture, Hardware & Software requirements

Deliverables for Phase – 2 Evaluation: Demonstration, Project report (adhering to the format)

Evaluation of Experiments

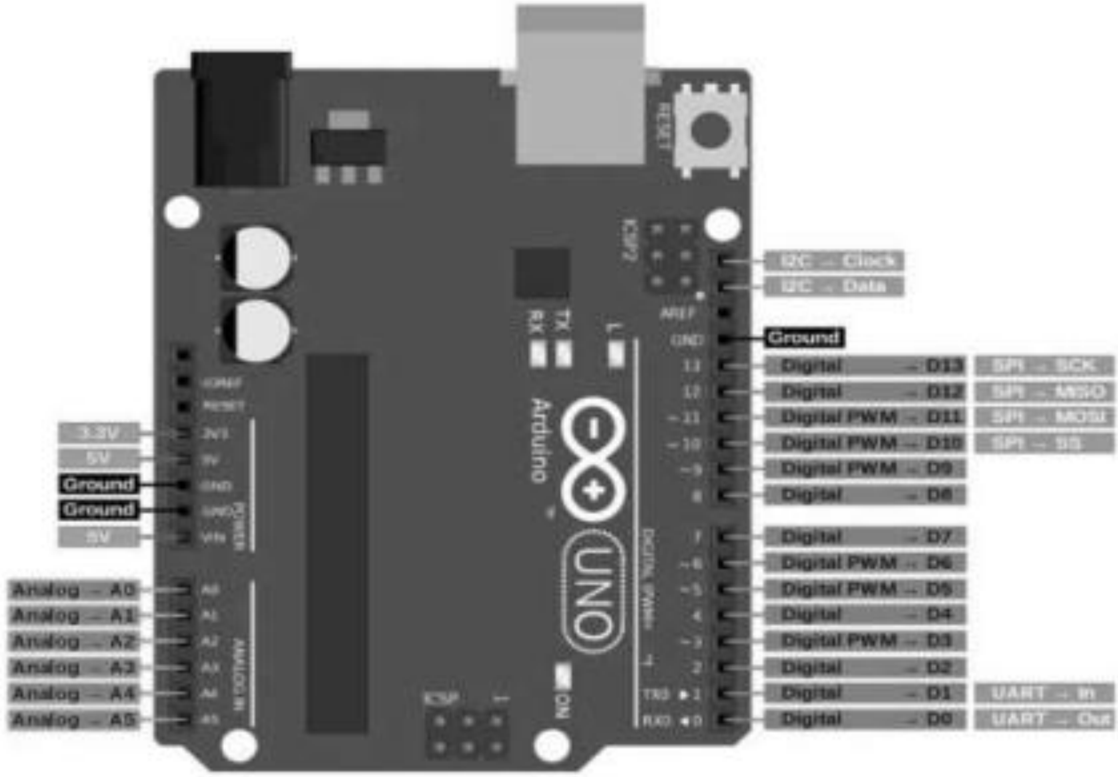
Program No.	Date of Submission	Lab Write-up and Execution marks (6 marks)			Viva voce Marks (4 marks)			Total Marks (10)	Signature
		a (2)	b (2)	c (2)	a (2)	b (1)	c (1)		
1									
2									
3									
4									
5									
6									
7									
8									
Total Marks (80)									
Reduced Marks (20)									

Project Evaluation

Project review	Date of Submission	a (5)	b (10)	c (10)	Total Marks (25)	Faculty Signature
Phase - 1						
Phase - 2						
Total Marks (50)						
Reduced Marks (20)						

LAB CIE		
Experiments Evaluation	Max – 20	
Project evaluation	Max - 20	
Lab test	Max - 10	
TOTAL	**Max – 50**	
Signature of the faculty		

Arduino Pin configuration layout:



Additional Information on Arduino board:

NOTE: Do NOT use a power supply greater than 20 Volts as you will overpower (and thereby destroy) your Arduino. The recommended voltage for most Arduino models is between 6 and 12 Volts.

Pins (5V, 3.3V, GND, Analog, Digital, PWM, AREF)

The pins on your Arduino are the places where you connect wires to construct a circuit (probably in conjuction with a breadboard and some wire. They usually have black plastic 'headers' that allow you to just plug a wire right into the board. The Arduino has several different kinds of pins, each of which is labeled on the board and used for different functions.

· **GND (3)**: Short for 'Ground'. There are several GND pins on the Arduino, any of which can be used to ground your circuit.

· **5V (4) & 3.3V (5)**: As you might guess, the 5V pin supplies 5 volts of power, and the 3.3V pin supplies 3.3 volts of power. Most of the simple components used with the Arduino run happily off of 5 or 3.3 volts.

· **Analog (6)**: The area of pins under the 'Analog In' label (A0 through A5 on the UNO) are Analog In pins. These pins can read the signal from an analog sensor (like a temperature sensor) and convert it into a digital value that we can read.

· **Digital (7)**: Across from the analog pins are the digital pins (0 through 13 on the UNO). These pins can be used for both digital input (like telling if a button is pushed) and digital output (like powering an LED).

· **PWM (8)**: You may have noticed the tilde (~) next to some of the digital pins (3, 5, 6, 9, 10, and 11 on the UNO). These pins act as normal digital pins, but can also be used for something called Pulse-Width Modulation (PWM). We have a tutorial on PWM, but for now, think of these pins as being able to simulate analog output (like fading an LED in and out).

· **AREF (9)**: Stands for Analog Reference. Most of the time you can leave this pin alone. It is sometimes used to set an external reference voltage (between 0 and 5 Volts) as the upper limit for the analog input pins.

Reset Button

Just like the original Nintendo, the Arduino has a reset button **(10)**. Pushing it will temporarily connect the reset pin to ground and restart any code that is loaded on the Arduino. This can be very useful if your code doesn't repeat, but you want to test it multiple times. Unlike the original Nintendo however, blowing on the Arduino doesn't usually fix any problems.

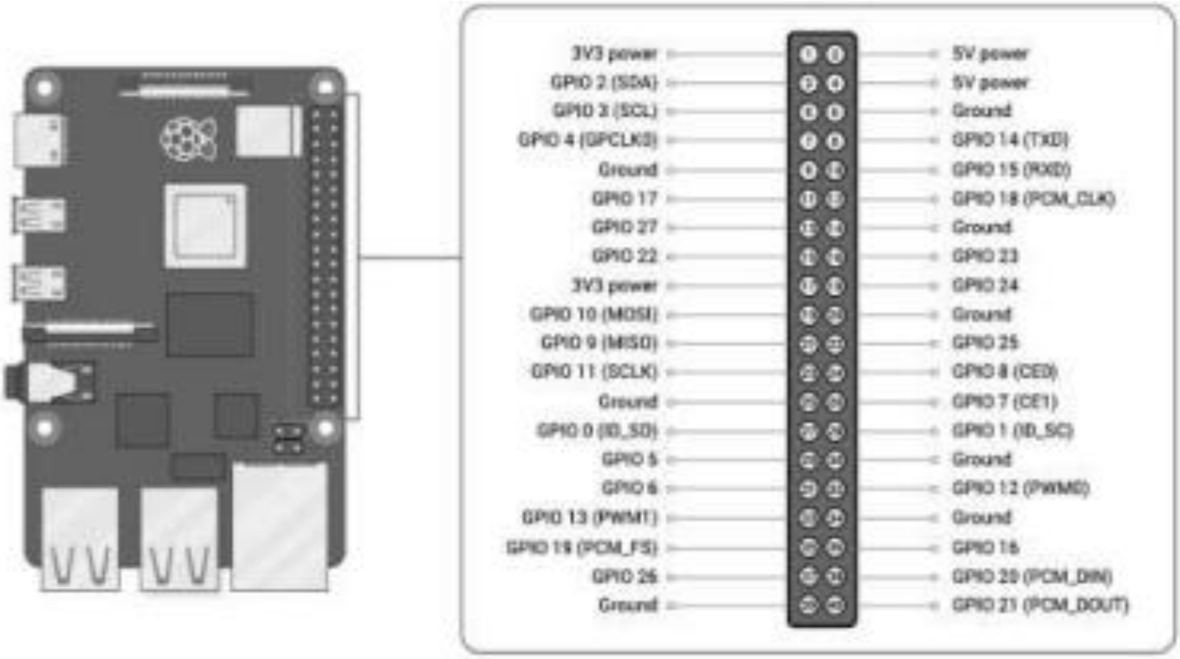
Power LED Indicator

Just beneath and to the right of the word "UNO" on your circuit board, there's a tiny LED next to the word 'ON' **(11)**. This LED should light up whenever you plug your Arduino into a power source. If this light doesn't turn on, there's a good chance something is wrong. Time to re check your circuit!

TX RX LEDs

TX is short for transmit, RX is short for receive. These markings appear quite a bit in electronics to indicate the pins responsible for serial communication. In our case, there are two places on the Arduino UNO where TX and RX appear – once by digital pins 0 and 1, and a second time next to the TX and RX indicator LEDs **(12)**. These LEDs will give us some nice visual indications whenever our Arduino is receiving or transmitting data (like when we're loading a new program onto the board).

Raspberry Pi Layout and Pin configuration details:



Additional Information on Raspberry Pi board:

Note: The GPIO pin numbering scheme is not in numerical order. GPIO pins 0 and 1 are present on the board (physical pins 27 and 28), but are reserved for advanced use.

Voltages:

Two 5V pins and two 3.3V pins are present on the board, as well as a number of ground pins (GND), which can not be reconfigured. The remaining pins are all general-purpose 3.3V pins, meaning outputs are set to 3.3V and inputs are 3.3V-tolerant.

Outputs:

A GPIO pin designated as an output pin can be set to high (3.3V) or low (0V).

Inputs:

A GPIO pin designated as an input pin can be read as high (3.3V) or low (0V). This is made easier with the use of internal pull-up or pull-down resistors. Pins GPIO2 and GPIO3 have fixed pull-up resistors, but for other pins this can be configured in software.

Other functions:

As well as simple input and output devices, the GPIO pins can be used with a variety of alternative functions, some are available on all pins, others on specific pins.

· **PWM (pulse-width modulation)**

- Software PWM available on all pins
    - Hardware PWM available on GPIO12, GPIO13, GPIO18, GPIO19

· **SPI**

- SPI0: MOSI (GPIO10); MISO (GPIO9); SCLK (GPIO11); CE0 (GPIO8), CE1 (GPIO7)
- SPI1: MOSI (GPIO20); MISO (GPIO19); SCLK (GPIO21); CE0 (GPIO18); CE1 (GPIO17); CE2 (GPIO16)

· **I2C**

- Data: (GPIO2); Clock (GPIO3)
- EEPROM Data: (GPIO0); EEPROM Clock (GPIO1)

· **Serial**

- TX (GPIO14); RX (GPIO15)

GPIO pinout

A GPIO reference can be accessed on your Raspberry Pi by opening a terminal window and running the command pinout. This tool is provided by the GPIO Zero Python library, which is installed by default in Raspberry Pi OS.

Warnings/Instruction for Circuit Rig up:

- While connecting simple components to the GPIO pins is perfectly safe, it's important to be careful how you wire things up.
- LEDs should have resistors (3kΩ) to limit the current passing through them.
  · Do not use 5V for 3.3V components.
- Do not connect motors directly to the GPIO pins, instead use an H-bridge circuit or a motor controller board.
- If you are getting under voltage detected message, means you are not supplying enough power to the Pi. Use a better USB charger. Better to use the official RPi charger. · Do proper connections and don't lead to shot circuit conditions.

Experiment 1

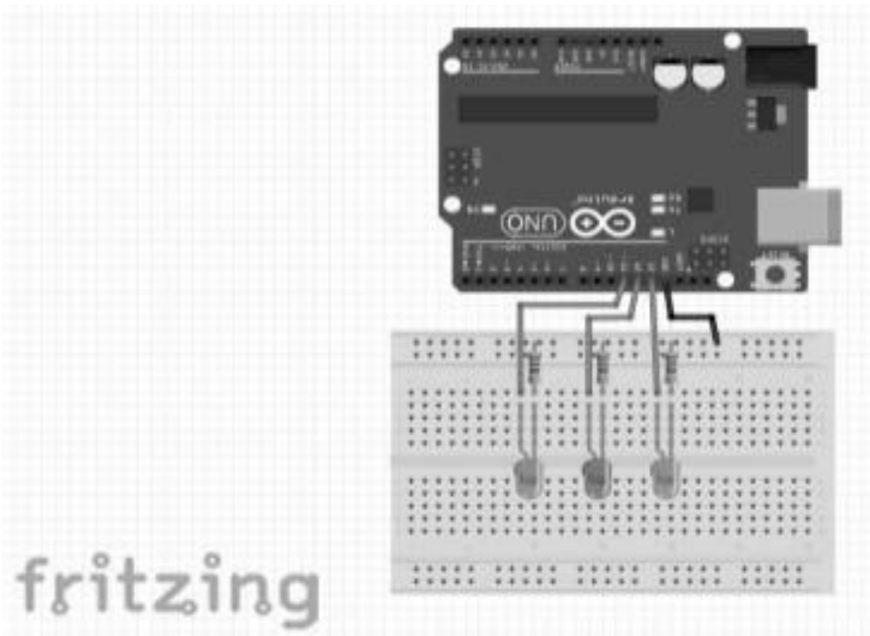
Title of the Experiment:

Set up a timer and delay for each of the three LED lights using Arduino

Components Required:

· Arduino Uno
· 3 LEDs
· 3 resistors (220 ohm)
· Breadboard
· Jumper wires

Circuit Design:



Pin Configuration & Connection Explanation:

- Connect the anode side of each LED to Arduino pins 11, 12, and 13
- Connect the ground (GND) of the Arduino to the breadboard.
- Connect the cathode end of each LED to a separate resistor (220 ohm).
- Connect the other end of each resistor to the ground rail of the breadboard.

Outcome Expected:

Upon completing this experiment, students should be able to:

- Set up a timer and delay for each LED using Arduino.
- Understand the role of delays in controlling the timing of LED operations.
- Execute sequential LED operations with different time intervals.
- Gain practical experience in programming Arduino for IoT applications involving timing and synchronization.

Purpose of Experiment:

Program Script used:

Actual Outcome:

Inference/s (Observation):

Experiment 2

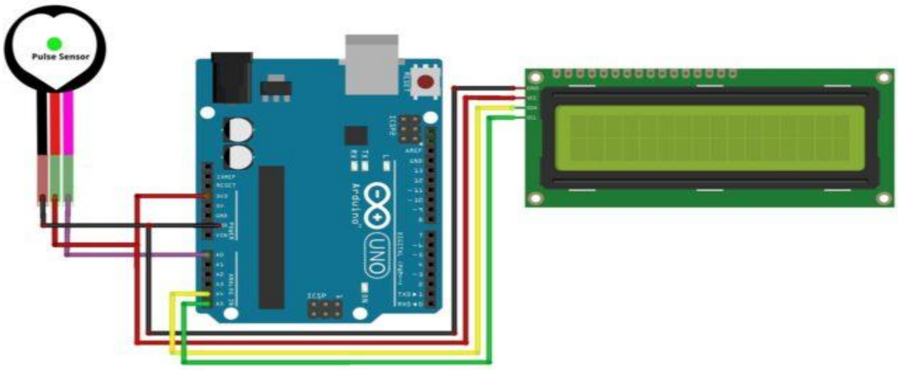
Title of the Experiment:

Interface **Pulse Sensor** with **Arduino** to Measure **Pulse Rate (BPM)** or **Heart Beat** value. Pulse rate will be displayed on 16×2 **LCD Display**

Components Required:

- Arduino Uno
- Pulse Sensor (SEN-11574)
- 16 X 2 LCD display
- Breadboard
- Jumper wires

Circuit Design:



Pin Configuration & Connection Explanation:

- Connecting the Pulse Sensor to an Arduino is a breeze.
- Connect three wires: two for power and one for reading the sensor value. The module can be supplied with either 3.3V or 5V. Positive voltage is connected to '+,' while ground is connected to '-.' The third 'S' wire is the analog signal output from the sensor, which will be connected to the Arduino's A0 analog input.
- Connect the VCC pin on the I2C LCD to the 5V pin on the Arduino.
- Connect the GND pin on the I2C LCD to the GND pin on the Arduino.
- Connect the SCL pin on the I2C LCD to the SCL pin (A5) on the Arduino.
- Connect the SDA pin on the I2C LCD to the SDA (A4) pin on the Arduino

Outcome Expected:

Place your finger on Pulse Sensor and you may see the BPM Value displayed on LCD Screen.

Purpose of Experiment:

Program Script used:

Actual Outcome:
Inference/s (Observation):

Experiment 3

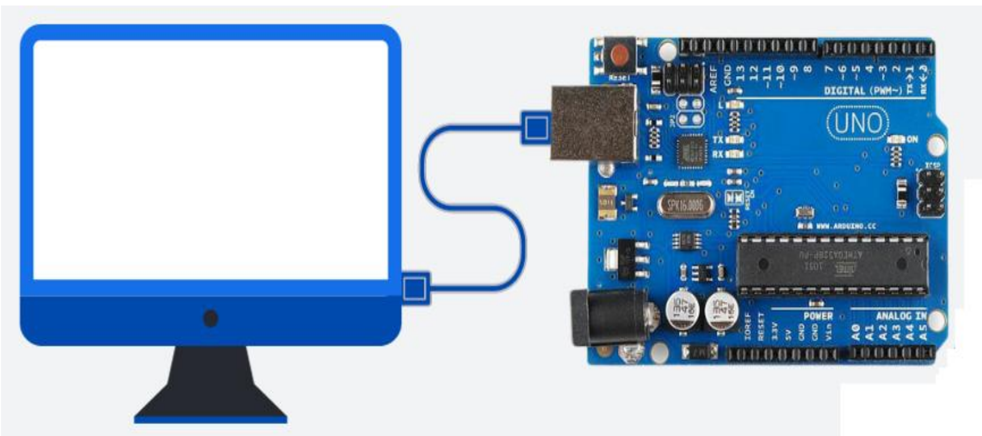
Title of the Experiment:

Interface PC with Arduino Uno using Serial Communication Port to capture different kinds of information on **serial monitor**, including system stats like network utilization, CPU load and disk space.

Components Required:

- Arduino UNO
- USB Cable
- Computer with USB port

Circuit Design:



Pin Configuration & Connection Explanation:

- Arduino UNO board is connected to the PC via a serial USB cable which is connected to the USB port of the PC

Outcome Expected:

Display information on **serial monitor** having system stats like network utilization, CPU load and disk space. Also show the change on serial monitor whenever there is change in any of the system stats.

Purpose of Experiment:

Program Script used:

Actual Outcome:

Inference/s (Observation):

Experiment 4

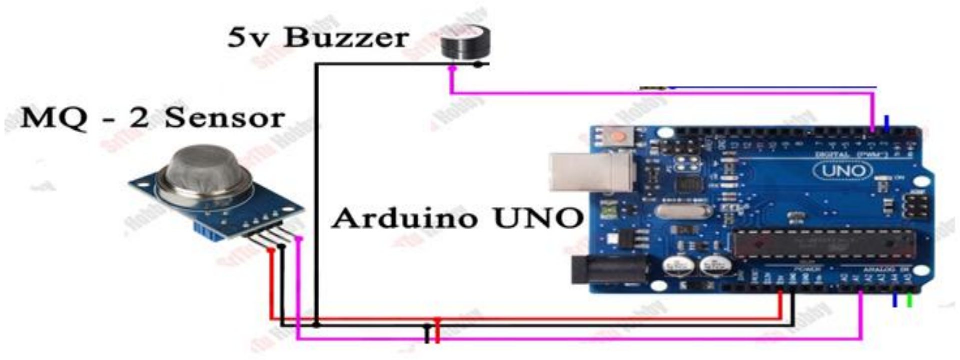
Title of the Experiment:

Interface **gas detector** with Arduino to switch on the **buzzer** on gas leakage.

Components Required:

- MQ-2 Gas Sensor
- Arduino Uno
- Active Buzzer (5V)
- Jumper wires

Circuit Design:



Pin Configuration & Connection Explanation:

- Connect the VCC (power) pin of the MQ2 sensor to the 5V pin on the Arduino.
- Connect the GND pin of the MQ2 sensor to the GND pin on the Arduino.
- Connect the analog signal (A0 pin) from the MQ2 sensor to an analog pin A0 on the Arduino.
- Connect the 5V active buzzer (+) to the digital pin 13 of the Arduino board.
- Connect the 5V active buzzer (-) to the digital GND pin of the Arduino board.

Outcome Expected:

Gas leakage detection system using the MQ-2 sensor, Arduino UNO board, and an active buzzer should present a comprehensive and effective solution for monitoring and alerting against potential gas leak.

Purpose of Experiment:

Program Script used:

Actual Outcome:
Inference/s (Observation):

Experiment 5

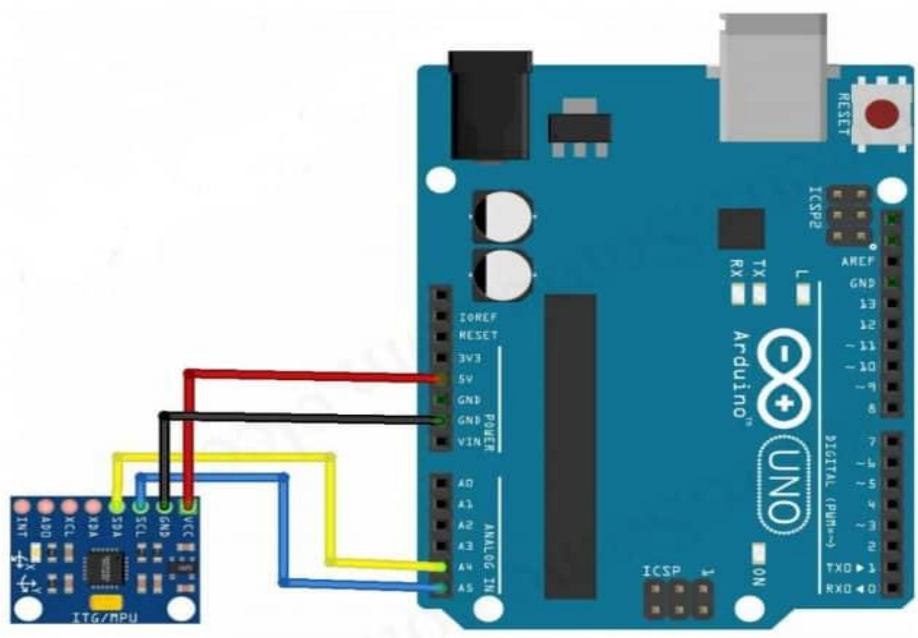
Title of the Experiment:

Demonstrate the use of **accelerometer** by measuring the angle of tilt on **serial monitor** based on object motion with Arduino

Components Required:

- MPU-6050 Accelerometer Gyroscope
- Arduino (read Best Arduino Starter Kits)
- Breadboard
- Jumper wires

Circuit Design:



Pin Configuration & Connection Explanation:

- Connect VCC of MPU6050 to 5V on Arduino
- Connect GND of MPU6050 to GND on Arduino
- Connect SDA of MPU6050 to A4 on Arduino
- Connect SCL of MPU6050 to A5 on Arduino

Outcome Expected:

Display the Roll and Pitch values through the serial port on serial monitor to identify the angle of object tilt.

Purpose of Experiment:

Program Script used:

Actual Outcome:

Inference/s (Observation):

Experiment 6

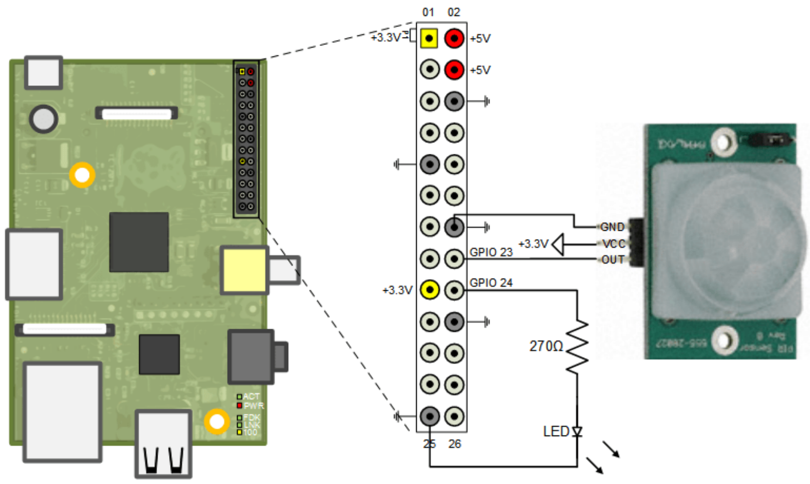
Title of the Experiment:

Demonstrate object detection using **PIR sensor** and **LED interfacing** with Raspberry-Pi

Components Required:

- Raspberry Pi
- PIR Sensor
- Breadboard
- LED
- Wires

Circuit Design:



Pin Configuration & Connection Explanation:

- The PIR motion sensor has three pins: VCC, GND, and Data. Connect VCC to the 3V3 pin, GND to a GND pin, and the Data pin to a suitable Raspberry Pi GPIO (GPIO 18)

Outcome Expected:

Move your hand in front of the PIR motion sensor. One should get a message on the shell mentioning that “motion was detected. After a few seconds”, if it’s no longer detecting motion, message saying that “motion stopped” should be displayed on shell.

Purpose of Experiment:

Program Script used:

Actual Outcome:
Inference/s (Observation):

Experiment 7

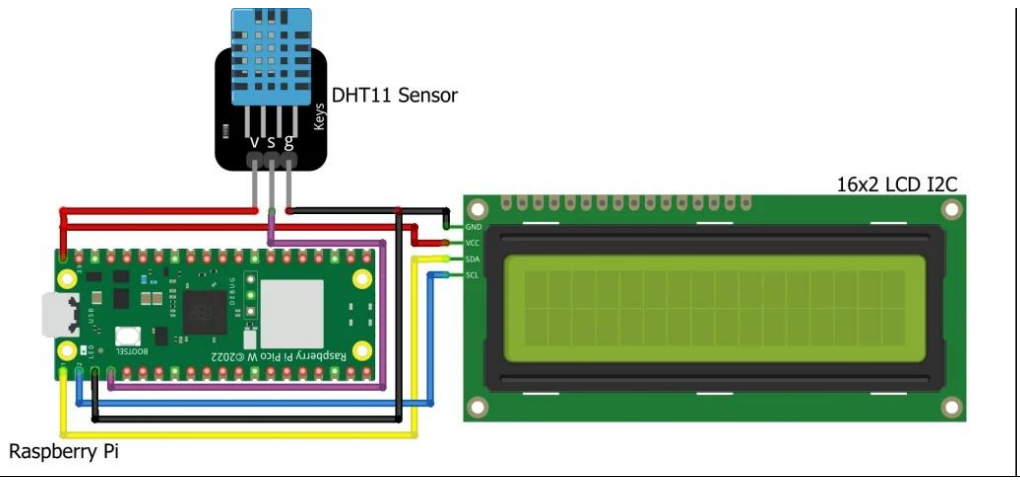
Title of the Experiment:

Demonstrate the measuring of Temperature & Humidity of weather using the **DHT11 Sensor** and display on 16 X 2 LCD display interfacing with the Raspberry-Pi. Also share the data using webserver at the same time on the browser.

Components Required:

- Raspberry Pi
- DHT11 Sensor
- 16×2 LCD Display (I2C)
- Breadboard and jumper wires
- Micro USB cable for power

Circuit Design:



Pin Configuration & Connection Explanation:

- Connect the DHT11 sensor to the Raspberry Pi and the 16×2 LCD display

Raspberry Pi	DHT11 Sensor	16x2 LCD Display
Vcc (PIN 40)	VCC	VCC
GND (PIN 3)	GND	GND
SDA GPIO 0 (PIN 1)		SDA
SCL GPIO 1 (PIN 2)		SCL
GPIO 2 (PIN 4)	Data Pin	

Outcome Expected:

After the code upload, LCD will start displaying the humidity and temperature. Web browser should also display the same information through the request send to web server.

Purpose of Experiment:

Program Script used:

Actual Outcome:

Inference/s (Observation):

Experiment 8

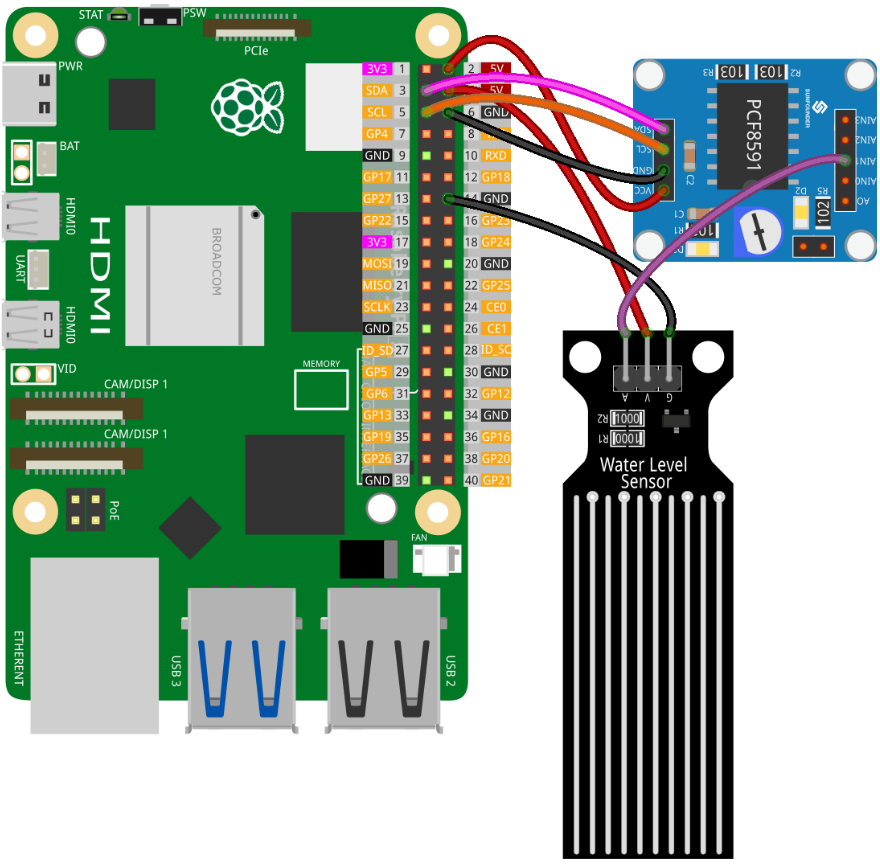
Title of the Experiment:

Water level sensing and sending water level information to **email** using **SMTP library** on Raspberry-Pi.

Components Required:

- Raspberry Pi
- Water Level Sensor Module
- Breadboard
- ADC DAC Converter Module
- https://randomnerdtutorials.com/raspberry-pi-send-email-python-smtp-server/ (Tutorial to send email using python – SMTP Server)

Circuit Design:



Pin Configuration & Connection Explanation:

The water level sensor has 3 pins:

The S (Signal) pin: This is an analog output that should be connected to one of the analog inputs on your Raspberry Pi.

The + (VCC) pin: This supplies power for the sensor and it is recommended to use between 3.3V – 5V.

The - (GND) pin: This is a ground connection.

- The MCP3002 (ADC-DAC converter) is connected using Pi's hardware SPI GPIO pins.
- D_{IN} goes to GPIO 10 (MOSI: master out slave in).
- D_{OUT} goes to GPIO 9 (MISO: master in slave out).
- CLK goes to GPIO 11 (SCK: serial clock).
- CS goes to GPIO 8 (chip enable zero often called chip select).
- VDD pin goes to 3.3 V on the Pi and the VSS goes to a ground pin.

Outcome Expected:

Read from a water level sensor using a Raspberry Pi and send the water level to the email-id received from the user using SMTP library

Purpose of Experiment:

Program Script used:

Actual Outcome:

Inference/s (Observation):

Experiment 9

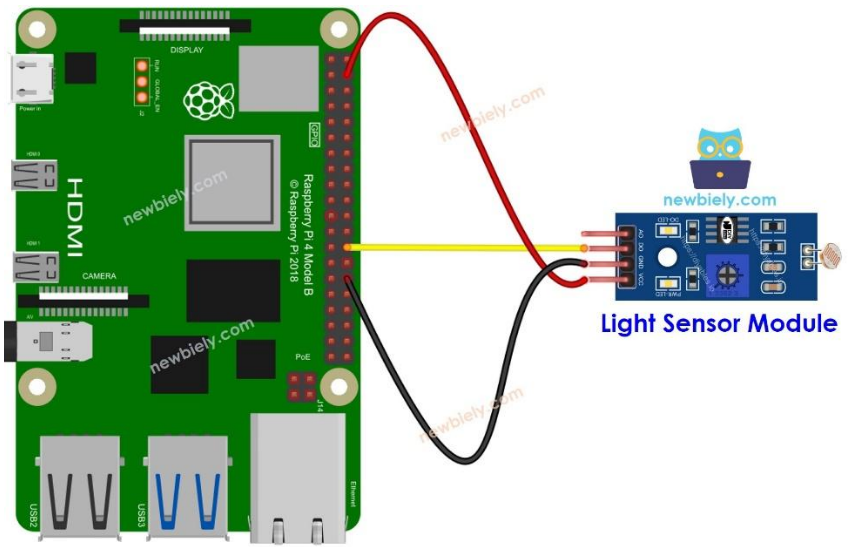
Title of the Experiment:

Develop a Smart lighting application using **LDR sensor** interfaced with Raspberry-Pi, **connected to cloud** for usage analysis.

Components Required:

- Raspberry Pi (any model with GPIO support)
- LDR (Light Dependent Resistor)
- 1μF Capacitor
- LED
- Resistor (220Ω for LED protection)
- Breadboard and Jumper Wires

Circuit Design:



Pin Configuration & Connection Explanation:

The LDR light sensor module has four pins with different functions:

VCC pin: Connect it to a power source (between 3.3V to 5V).

GND pin: Connect it to the ground (0V).

DO pin: This is a digital output pin. It goes **HIGH** when it's dark and **LOW** when it's light. You can adjust the threshold between darkness and lightness using a built-in potentiometer.

AO pin: This is an analog output pin. The output value decreases as the light gets brighter and increases as the light gets darker.

Connect your PC to the Raspberry Pi via SSH using the built-in SSH client on Linux and macOS or PuTTY on Windows.

Tutorial for sending data to cloud:
https://www.circuitbasics.com/logging-sensor-data-to-the-cloud-using-the-raspberry-pi/

Outcome Expected:

Capture the date and time whenever there is a change noticed on LDR and log the data to cloud. Display the data captured on cloud using visualization tools.

Purpose of Experiment:

Program Script used:

Actual Outcome:

Inference/s (Observation):

Experiment 10

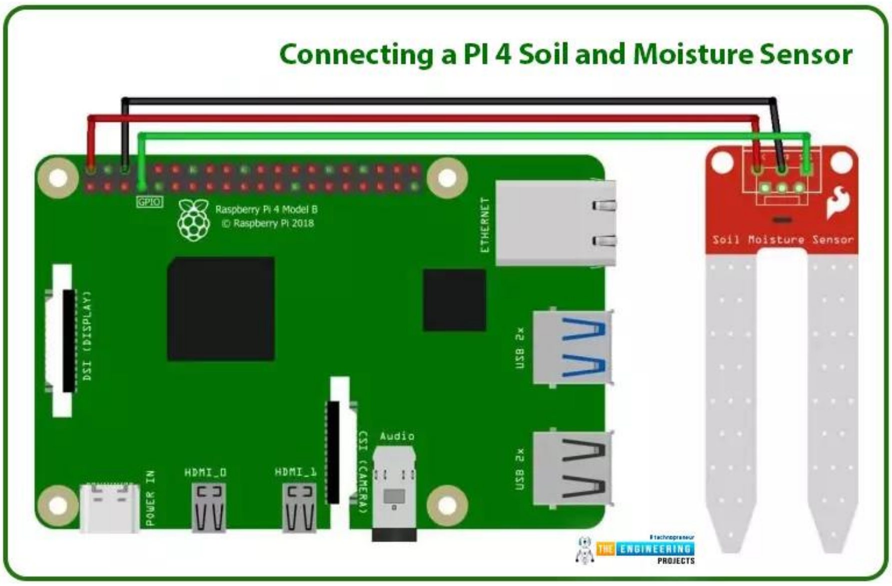
Title of the Experiment:

Demonstrate the use of **soil moisture sensor** by alerting the user through a **mail/message** based on moisture level with Raspberry-Pi

Components Required:

- Raspberry Pi
- Soil moisture sensor
- Breadboard
- Wires

Circuit Design:



Pin Configuration & Connection Explanation:

- Connect VCC pin of sensor to 5V of R-Pi
- Connect GND pin of sensor to GND of R-Pi
- Connect DATA pin of sensor to GPIO4 of R-Pi

https://circuitdigest.com/microcontroller-projects/raspberry-pi-phone-by-interfacing-gsm-module

Outcome Expected:

Once the humidity level is detected above threshold, alert the user through a mail/message.

Purpose of Experiment:

Program Script used:

Actual Outcome:

Inference/s (Observation):

Innovative experiments using Arduino and Raspberry-Pi

1. Autonomous firefighting robot
2. Automated solar grass cutter
3. IoT based smart energy meter
4. Smart plant monitoring and watering system
5. Gesture controlled appliance system
6. Smart dustbin with IoT notifications
7. Posture tracking system
8. Arduino-based gaming glove
9. Automatic sketching machine
10. Electronic eye controlled security system
11. Sun tracking solar panel
12. Fluid flow rate monitor
13. Automatic noise level monitor
14. Whistle detector switch
15. Fingerprint door lock
16. Pet feeder
17. Stair climbing robot
18. Self balancing robot
19. Earthquake detector
20. Non-contact infrared thermometer

Sample IoT Project topics

1. Remote Health Monitoring System for old aged Patients – IoT in Healthcare
2. Regular monitoring of patient Heart Rate and SpO2
3. Wrong Posture Muscle Strain Detector
4. Safety Monitoring System for Manual Wheelchairs
5. Smart Door Lock System – IoT Home Automation
6. Find Out When Someone Takes Your Stuff – IoT Home Security
7. Smoke Detecting IoT Device Using Gas Sensor – IoT Home Safety
8. Remote Plant Monitor – IoT Home Automation
9. Gesture-Controlled Contactless Switch for Smart Home
10. Push up Counter - IoT in Fitness
11. Mining Worker Safety Helmet
12. IoT-based Liquid Level Monitoring System
13. Contactless Doorbell
14. IoT-Based Automated Table Lamp
15. Control TV With Alexa – IoT TV Controller
16. Noise Detector with Automatic Recording System
17. Smart Mirror
18. Weather Reporting System based on Raspberry pi
19. IOT based Manhole Detection and Monitoring System
20. IOT Garbage Segregator & Bin Level Indicator
21. IoT-enabled Secure Home Automation
22. IoT-based Smart Gardening System
23. IoT-enabled Elderly Care System
24. Smart systems to bring equality in governance
25. Monitoring community development through IoT technologies